

AdsLyve Media — Website & Admin Panel Documentation

1. Project Overview

AdsLyve Media is a modern digital marketing agency website built with **Next.js** and supported by a secure admin management system.

The project consists of two major parts:

- Public Website** — accessible to all visitors.
- Protected Admin Panel** — used to manage website content, leads, contact information, SEO settings, and administrator credentials.

The application uses a database-driven architecture, allowing important website content to be updated from the admin panel without directly modifying the source code.

2. Technology Stack

Technology	Purpose
Next.js	Frontend, routing and API development
Next.js API Routes	Backend REST API
MongoDB	Database
JWT	Admin authentication
bcryptjs	Password hashing
Axios	API requests
React Hook Form	Form handling and validation
Tailwind CSS	UI styling
Framer Motion	Animations and transitions
Sonner	Toast notifications
Lucide React	Icons

3. Application Structure

The application is divided into:

Public Website

The public website is available at:

/

It contains:

- Navbar

- Hero section
- About section
- Trusted Growth Partner section
- Services
- Industries We Serve
- Results That Matter
- FAQ
- Contact section
- Footer
- Lead enquiry popup
- WhatsApp/contact functionality

Admin Panel

The admin panel is protected and includes:

/admin/login

/admin/dashboard

/admin/about

/admin/services

/admin/leads

/admin/contact

/admin/settings

Only authenticated administrators should be able to access the protected admin pages.

4. Public Website

4.1 Home Page

Route

/

The home page is the primary public-facing page of the website.

Main Sections

Navbar

Provides navigation to the main sections of the website.

Hero

Introduces AdsLyve Media and communicates the agency's primary value proposition.

About

Provides information about the agency and its business.

Trusted Growth Partner

Highlights AdsLyve Media as a growth partner for businesses.

Services

Displays the services offered by the agency.

Services can be managed dynamically through the admin panel.

Industries We Serve

Displays the industries and business categories targeted by the agency.

Results That Matter

Highlights the agency's performance and growth-focused approach.

FAQ

Provides answers to frequently asked questions.

Contact

Displays contact information managed from the admin panel.

Footer

Contains website information, navigation, social links and copyright information.

5. Admin Panel

5.1 Admin Login

Route

/admin/login

The login page allows the administrator to authenticate using:

- Email
- Password

After successful authentication, the administrator can access the admin dashboard.

Authentication is handled using **JWT-based authentication**.

6. Admin Dashboard

Route

/admin/dashboard

The dashboard acts as the central administration area.

It provides access to the major management modules:

- About Management
- Services Management
- Leads Management

- Contact Management
- Website Settings

The dashboard also provides an overview of the website's managed content and enquiries.

7. About Management

Route

/admin/about

This section allows the administrator to manage the website's About content.

The administrator can edit the existing About section information from the admin panel.

Changes are stored in MongoDB and can then be reflected on the public website.

Typical Workflow

1. Open **Admin → About**.
 2. Edit the required information.
 3. Submit/save the changes.
 4. The updated information is stored in the database.
 5. The public website displays the updated content.
-

8. Services Management

Route

/admin/services

The Services Management section allows the administrator to maintain the services displayed on the website.

Available Operations

- Add service
- Edit service
- Delete service
- View existing services

This allows the agency to update its service offerings without changing the frontend source code.

Typical Workflow

Add

1. Open Services Management.
2. Enter service information.
3. Submit the form.
4. Service is stored in MongoDB.

Edit

1. Select an existing service.
2. Update the required information.
3. Save the changes.

Delete

1. Select the service.
2. Choose delete.
3. Confirm the deletion.

9. Leads Management

Route

/admin/leads

The Leads Management section handles enquiries submitted through the website.

Leads can originate from the website enquiry/consultation form.

Available Operations

- View leads
- View lead details
- Manage lead status
- Delete leads

The lead information is stored in MongoDB.

Lead Flow

Website Visitor

↓

Lead Form

↓

POST /api/admin/leads

↓

MongoDB

↓

Admin Leads Panel

The administrator can review enquiries and update their status from the admin panel.

10. Contact Information Management

Route

/admin/contact

This section manages the contact information displayed throughout the public website.

The administrator can update the configured contact details from the admin panel.

Once updated, the information can be consumed by website components such as the Contact section and Footer.

11. Website Settings

Route

/admin/settings

Website Settings contains two major categories of configuration.

SEO Settings

The administrator can manage:

- SEO Meta Title
- SEO Meta Description

These values are used for the website's metadata.

Footer Settings

The administrator can manage:

- Footer Copyright Text

Admin Account Settings

The administrator can update:

- Admin Name
- Admin Email
- Current Password
- New Password
- Confirm New Password

Password changes require the current password and appropriate validation.

12. Environment Configuration

Create an environment file in the project root:

.env.local

Add the following variables:

MONGODB_URI=MongoDB_Uri

JWT_SECRET=jwt-secret-key

NEXT_PUBLIC_APP_URL=your-public-site-url

Environment Variables

MONGODB_URI

MongoDB connection string used by the application to connect to the database.

Example:

MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/database

JWT_SECRET

Secret key used for JWT authentication.

Example:

JWT_SECRET=your-long-random-secret

Use a strong, private value in production.

NEXT_PUBLIC_APP_URL

The public URL of the deployed website.

Example:

NEXT_PUBLIC_APP_URL=https://adslyvemedia.com

13. Database Configuration

The application uses **MongoDB** for persistent data storage.

The database connection is configured through:

MONGODB_URI

The application uses a database connection utility to establish the MongoDB connection before performing database operations.

The database stores information required by the application, including administrator information and website-managed content.

14. Initial Admin Account Setup

Before using the admin panel for the first time, an administrator account needs to be created.

The project provides a dedicated setup endpoint:

POST /api/admin/setup

Request Body

```
{  
  "name": "Admin Name",  
  "email": "admin@example.com",  
  "password": "your-password"  
}
```

Password Requirement

The initial password must contain at least:

8 characters

The password is hashed using bcryptjs before being stored in MongoDB.

Important

The setup API checks whether an administrator already exists.

If an admin account already exists, the endpoint returns:

Admin already exists.

Therefore, the setup endpoint is intended for **initial administrator creation**, not routine account creation.

15. Recommended Initial Setup Process

Follow this sequence when setting up the project for the first time:

Step 1 — Install Dependencies

Install the project's dependencies using the package manager configured for the project.

Step 2 — Configure Environment Variables

Create:

`.env.local`

and configure:

`MONGODB_URI=...`

`JWT_SECRET=...`

`NEXT_PUBLIC_APP_URL=...`

Step 3 — Configure MongoDB

Make sure the MongoDB database is accessible from the environment where the application is running.

Step 4 — Start the Application

Run the project's development server.

Step 5 — Create Initial Admin

Send a POST request to:

`/api/admin/setup`

with the administrator's:

- Name
- Email
- Password

Step 6 — Login

Open:

/admin/login

and sign in using the newly created credentials.

Step 7 — Configure Website

From the admin panel, configure:

- About content
- Services
- Contact information
- SEO title
- SEO description
- Footer copyright
- Admin credentials

16. Authentication API

Login

POST /api/auth/login

Authenticates an administrator using email and password.

Logout

POST /api/auth/logout

Logs the administrator out of the authenticated session.

Current Admin

GET /api/auth/me

Returns information about the currently authenticated administrator.

This is used to determine the current admin profile and authentication state.

17. About API

Get About Content

GET /api/admin/about

Retrieves the current About section data.

Update About Content

PUT /api/admin/about

Updates the About section information.

18. Contact API

Get Contact Information

GET /api/admin/contact

Retrieves the current contact information.

Update Contact Information

PUT /api/admin/contact

Updates the contact information.

19. Leads API

Get Leads

GET /api/admin/leads

Retrieves website leads/enquiries.

Create Lead

POST /api/admin/leads

Creates a new lead.

This endpoint is used by the public enquiry form.

Update Lead Status

PATCH /api/admin/leads

Updates the status of an existing lead.

Delete Lead

DELETE /api/admin/leads?id=id

Deletes a specific lead.

Example:

/api/admin/leads?id=65abc123...

20. Services API

Get Services

GET /api/admin/services

Retrieves all services.

Create Service

POST /api/admin/services

Creates a new service.

Update Service

PUT /api/admin/services

Updates an existing service.

Delete Service

DELETE /api/admin/services?id=id

Deletes a service using its MongoDB ID.

21. Admin Profile API

PUT /api/admin/profile

Used for updating administrator credentials.

Supported information includes:

- Name
- Email
- Current password
- New password
- Confirm password

Password changes are handled securely using password hashing.

22. Website Settings API

Get Settings

GET /api/admin/settings

Retrieves the current website settings.

Update Settings

PUT /api/admin/settings

Updates website configuration such as:

- SEO title
 - SEO description
 - Footer copyright
-

23. Complete API Reference

Method	Endpoint	Purpose
POST	/api/auth/login	Admin login
POST	/api/auth/logout	Admin logout
GET	/api/auth/me	Get authenticated admin
POST	/api/admin/setup	Create initial admin

Method	Endpoint	Purpose
GET	/api/admin/about	Get About data
PUT	/api/admin/about	Update About data
GET	/api/admin/contact	Get contact information
PUT	/api/admin/contact	Update contact information
GET	/api/admin/leads	Get leads
POST	/api/admin/leads	Create lead
PATCH	/api/admin/leads	Update lead status
DELETE	/api/admin/leads?id=id	Delete lead
PUT	/api/admin/profile	Update admin profile
GET	/api/admin/services	Get services
POST	/api/admin/services	Create service
PUT	/api/admin/services	Update service
DELETE	/api/admin/services?id=id	Delete service
GET	/api/admin/settings	Get website settings
PUT	/api/admin/settings	Update website settings

24. Admin Usage Guide

Login

Navigate to:

/admin/login

Enter the registered admin email and password.

After successful authentication, access the dashboard.

Managing About Content

Navigate to:

Admin → About

Edit the required content and save it.

Managing Services

Navigate to:

Admin → Services

Use the available controls to add, edit or delete services.

Managing Leads

Navigate to:

Admin → Leads

Review incoming enquiries and manage their status.

Managing Contact Information

Navigate to:

Admin → Contact

Update the contact details used by the public website.

Managing SEO and Footer

Navigate to:

Admin → Settings

Update:

- SEO title
- SEO description
- Copyright text

Updating Admin Account

From:

Admin → Settings

update the administrator's:

- Name
- Email
- Password

When changing the password, provide the current password and the new password confirmation.

25. Security

The application implements several security mechanisms for the administrator system:

- JWT-based authentication
- Protected admin routes
- Password hashing using bcryptjs
- Current-password verification for password changes
- Environment variables for sensitive configuration
- Authentication API for session verification
- Admin-only management APIs

The administrator password should never be stored as plain text.

26. Production Deployment Checklist

Before deploying the application:

- Configure the production MONGODB_URI.
- Generate a strong production JWT_SECRET.
- Set the correct NEXT_PUBLIC_APP_URL.
- Ensure MongoDB is accessible from the production server.
- Create the initial administrator account.
- Verify admin login.
- Verify admin logout.
- Test all protected admin routes.
- Test lead submission from the public website.
- Test lead management.
- Test About updates.
- Test service CRUD operations.
- Test contact updates.
- Test SEO settings.
- Test footer copyright updates.
- Test admin credential updates.
- Verify the production website metadata.
- Verify environment variables are not committed to source control.

27. Important Operational Notes

Environment File

Do not commit production secrets such as:

MONGODB_URI

JWT_SECRET

to a public repository.

Initial Admin Setup

The `/api/admin/setup` endpoint should only be used for creating the first administrator account. Once an admin exists, the API itself prevents another administrator from being created.

Password Management

Administrators should use a strong password and avoid sharing credentials.

Database

Regular MongoDB backups are recommended for protecting:

- Leads
- Services
- About content
- Contact information
- Admin data
- Website settings

28. Project Route Reference

Route	Access	Description
/	Public	Main website
/admin/login	Public	Administrator login
/admin/dashboard	Protected	Admin dashboard
/admin/about	Protected	About management
/admin/services	Protected	Service management
/admin/leads	Protected	Lead management
/admin/contact	Protected	Contact management
/admin/settings	Protected	SEO, footer and admin settings
